

Title of thesis Second title line

Master Thesis of

Name

At the Department of Informatics and Mathematics
Chair of Theoretical Computer Science



Reviewers: ?
 ?
Advisors: ?
 ?

Time Period: 21st August 2018 – 31st August 2018

Statement of Authorship

I hereby declare that this document has been composed by myself and describes my own work, unless otherwise acknowledged in the text.

Passau, July 7, 2025

Abstract

A short summary of what is going on here.

Deutsche Zusammenfassung

Kurze Inhaltsangabe auf deutsch.

Contents

1. Introduction	1
2. Preliminaries	3
3. Reduction of the equivalent classes	5
3.1. The induced Hannani-tutte from 2-SAT and planarity	5
3.2. Equivalent classes and Hannani-tutte	5
3.3. Reduction of the equivalent classes	5
3.3.1. Algorithm description	6
3.3.2. Correctness of the algorithm	6
4. Conclusion	11
Bibliography	13
Appendix	15
A. Appendix Section 1	15

1. Introduction

This chapter should contain

1. A short description of the thesis topic and its background.
2. An overview of related work in this field.
3. Contributions of the thesis.
4. Outline of the thesis.

2. Preliminaries

This chapter should provide the foundations of the thesis.

3. Reduction of the equivalent classes

The content chapters of your thesis should of course be renamed. How many chapters you need to write depends on your thesis and cannot be said in general.

3.1. The induced Hannani-tutte from 2-SAT and planarity

...

3.2. Equivalent classes and Hannani-tutte

3.3. Reduction of the equivalent classes

After linking the Equivalent Classes to the hanani-tutte drawing induced by the 2-Sat formula (insert paper), and also presenting the cases that lead to a cyclic relation between some vertices of the same level, we can now present our algorithm that reduce the set equivalent classes resulted from resolving the 2-SAT formula (insert papers).

Definition 3.1 (level assignement). *level assignement $l : V \rightarrow \{1, \dots, k\}$ with $k \in \mathbb{N}$ that satisfies $l(u) + 1 = l(v)$ for each edge $(u, v) \in E$*

Definition 3.2. *adj outputs the distinct endpoints in a level h of adjacent edges with the shared source endpoint v from $h - 1$.*

$$\begin{aligned} \text{adj}: (l(V), V) &\rightarrow \mathcal{P}(V) \\ (h, v) &\mapsto \{w \mid w \in V, (v, w) \in E \text{ and } l(w) = h\} \end{aligned}$$

Definition 3.3. *the function of distinct endpoint of adjacent edges $\mathcal{V}_{\text{adj}}$*

$$\begin{aligned} \mathcal{V}_{\text{adj}}: (l(V), V) &\rightarrow \mathcal{P}(\mathcal{V}) \\ (h, v) &\mapsto \{wx \mid w, x \in \text{adj}(v, h) \text{ and } w.\text{id} < x.\text{id}\} \end{aligned}$$

Definition 3.4 (confused vertex). *A confused vertex v is a vertex that is not adjacent to an incoming edge from the level $l(v) - 1$.*

We define $\mathcal{V}_{\text{confused}}$ as follows

$$\begin{aligned} \mathcal{V}_{\text{confused}}: (l(V)) &\rightarrow \mathcal{P}(\mathcal{V}) \\ h &\mapsto \{v \mid v \in V, l(v) = h, \text{ and } \forall u \in V l(u) = l(v) - 1 \text{ and } (u, v) \notin E\} \end{aligned}$$

Based on the (insert section proof that we can fix the issue with adjacent edges by making them equal eachother) we can fix the problem of cyclic relations between adjacent edges, to specify an order we suggest going with the index order of vertices, to do so we will need to concatenate the the equivalent classe of vw and it's .. wv (since we respect the consistency clause) With the equivalent class that we got from the algorithm based on (insert paper),

3.3.1. Algorithm description

We have as input a proper level graph proper level Graph $G = (V, E, l)$ with a level ordering l and an arbitrary index order and equivalent classes based on the 2-SAT formula of randerth (TODO put ref).

We will construct the graph from scartch for the second part of the algorithm. Let G' be an empty graph.

We traverse the graph level by level. Let the variable h be the current level. We look at the vertices v in the level h , we get the incident vertices from the upper incident level and lower incident levels separately. We work with the upper incident level vertices and lower one separately like the following , we give them an initial order, we get the equivalent class of the combinaisons of the relative orders one by one, if for the next equivalent class we have a relation in the opposite sense, we switch the order of the two vertices, at the end we will find an order with all the equivalent classes from the combination of the vertices with the new order having the same direction. we concatenate their equivalent classes and put the concatenation in all of their equivalent classes.

we update also the equivalent classes of the opposite order for synchronization.

we add v to G' and the edges incident to v with vertices from $G' - \{v\}$. if v is a cut-vertex in G' we also look for the connected comps in $G' - \{v\}$, we pick an arbitrary order of the connected comps (C_1, C_2) we then force the relation between a vertex w_1 of C_1 be the same for all the vertices in C_2 and that's by concatenating all the equivalent classes from the vertex w_1 to each vertex of C_2 and then making each equivalent class from it to equal the conatination of all of them. If C_2 encapsulate C_1 then we inverse the pair before forcing the relations.

3.3.2. Correctness of the algorithm

The algorithm is based on the fact that we can convert a 2-SAT truth assignement to a strong hanani-tutte drawing and that we can convert a strong hanani-tutte drawing to a planar drawing.

The assignement of equivalent classes represent the set of solutions from solving the 2-SAT assignement.

The algorithm reduce the set of truth assignement solutions to get rid of false positives and that by reducing the number of equivalent classes by concatening some of them, which lead to smaller number of possible solution that can lead to a level planar drawing.

Proof. Since we are just reducing the set of truth assignement that already satisfies 2-SAT, the planarity and the consistency clauses are still respected.

if we take a truth assignement ϕ from 2-sat solutions, we can induce a strong hanani-tutte drawing. The goal is to convert this strong hanani-tutte drawing into a x-montone embedding which is a planar drawing.

To convert the strong hanani-tutte drawing to weak hanani-tutte, we must remove any odd crossing of adjacent edges by making it an even crossing. We either have two edges

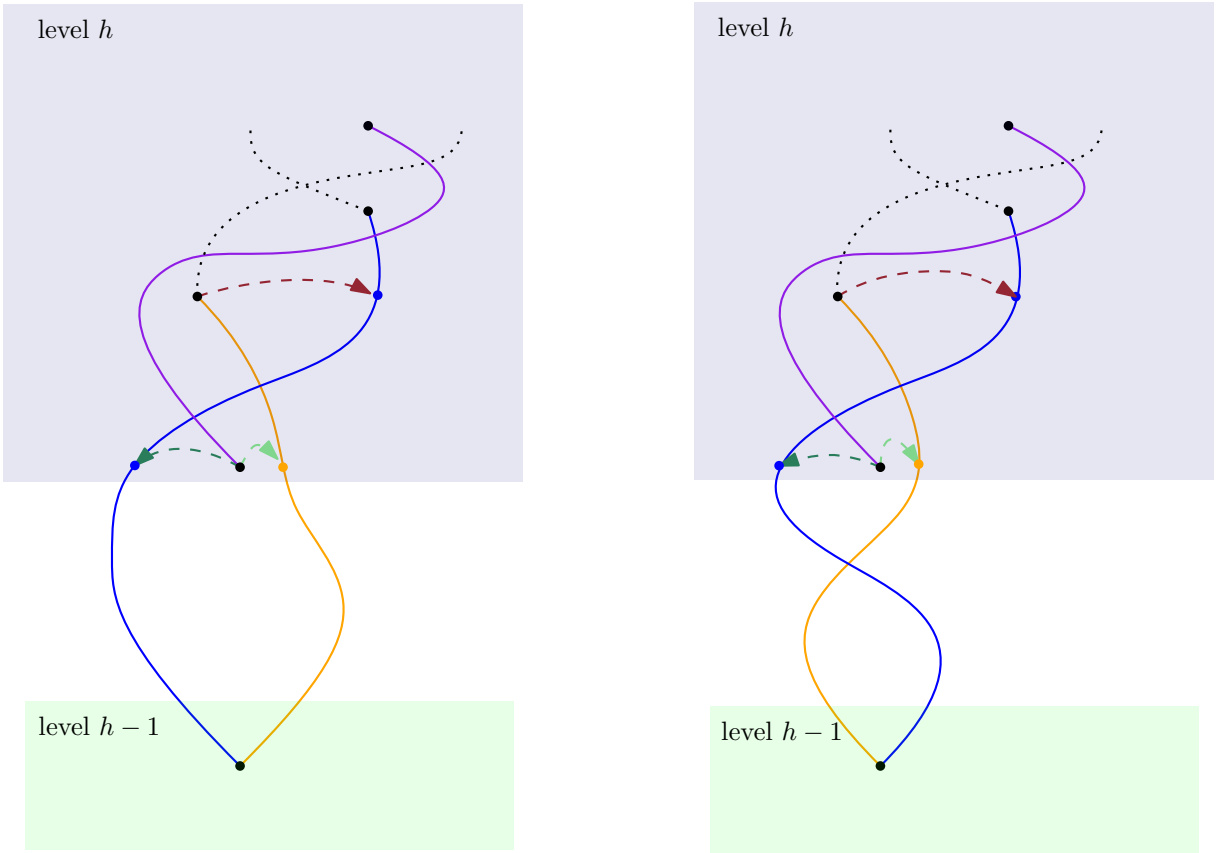


Figure 3.1.: consecutive edges in rotation that crosses oddly

consecutive in the rotation of the shared endpoint, so we can just cross them one more time near the shared endpoint, this doesn't change the order of the vertices in the hanani-tutte drawing Figure 3.1. The other cases are when there is at least an edge between the two edges that crosses oddly and this edge crosses evenly all edges of the constructed graph so far, we have three major cases (TODO describing the cases from the hanani-tutte to xmonotone paper) , and we can get rid the odd crossing by concatenating the equivalent classes while still respecting the consistency clause Figure 3.2

After the modification, each element of the set of solutions from the reduced equivalent classes will lead to a weak hanani-tutte drawing, there is a special case in weak hanani-tutte that doesn't lead to an equivalent x-monotone embedding wich means that we need to change the relative order of some vertices. We mimick the construction in the proof of theorem 2.1 (reference hanani-tutte to xmononte paper) of the graph G' (which equals G at the end of the construction) , when the algorithm reach the special case which is when the new added x_1 adds more than one edge to the graph G' and it is not connected to the last added vertex before it in G' , from the proof we can notice that we fix the order of two connected comps to each other if they don't have any relative order (TODO explain what's a relative order in this context), if they do, meaning that one encapsulate the other then each vertex of the encapsulating component forces that it has the same equivalent class to all vertices of the encapsulated comp. From the algorithm we take an arbitrary order of the components (C_1, C_2) and we force all elements of C_2 to have the same equivalent class with the same vertex from C_1 . If C_1 is the encapsulated component, we just switch C_1 with C_2 .

□

Algorithm 3.1: EQUIVALENCE CLASSES REDUCTION

Input: proper level Graph $G = (V, E, l)$ and Eq
Output: Eq

```

1  $\mathcal{V} = \{uw \mid u, w \in V, u \neq w, l(u) = l(w)\}$ 
   /* reducing the equivalence classes of adjacent edges */
2 forall  $h \in l(V)$  do
   /* i still need to make sure adding  $h + 1$  is valid here. */
3   forall  $v \in \{V \mid l(v) \in \{h - 1, h + 1\}\}$  do
4      $S \leftarrow \emptyset$ 
5      $\mathcal{V}_{\text{ord}} \leftarrow \emptyset$ 
6     forall  $uw \in \mathcal{V}_{\text{adj}}(h, v)$  do
7       if  $wu \in S$  then
8          $\mathcal{V}_{\text{ord}} \leftarrow \mathcal{V}_{\text{ord}} \cup \{wu\}$ 
9          $S \leftarrow S \cup \text{Eq}(wu)$ 
10      else
11         $\mathcal{V}_{\text{ord}} \leftarrow \mathcal{V}_{\text{ord}} \cup \{uw\}$ 
12         $S \leftarrow S \cup \text{Eq}(uw)$ 
13     $\forall uw \in \mathcal{V}_{\text{ord}}: \text{Eq}(uw) = \bigcup_{u', w' \in \mathcal{V}_{\text{ord}}} \text{Eq}(u'w')$ 
   /* Synchronize with the symmetrical order to  $\mathcal{V}_{\text{ord}}$  */
14     $\forall uw \in \mathcal{V}_{\text{ord}}: \text{Eq}(wu) = \{xv, vx \in \text{Eq}(uw)\}$ 
   /* we have equivalent classes that can induce a weak
      hanani-tutte drawing */
15   forall  $v$  in vertices of level  $h$  do
16      $G' \rightarrow G' \cup \{(w, v) \mid w \in V, (w, v) \in E \text{ and } l(w) = l(v) + 1\}$ 
17     if  $v$  is a cut vertex of  $G'$  then
18       Comps list of connected comps of  $G' - \{v\}$ 
19       we pick an arbitrary order of the list Comps
20       forall  $(C_1, C_2)$  in combinaisons of pairs of connected components in
         Comps with the respect to the specified order do
21         if  $C_1$  is encapsulated by  $C_2$  then  $(C_1, C_2) \leftarrow (C_2, C_1)$ 
22         /* we consider the vertices of the  $h$  level only */
          $\forall v \in C_1(h) \forall w \in C_2(h) : \text{Eq}(v, w) \leftarrow \bigcup_{w' \in C_2(h)} \text{Eq}(v, w')$ 

```

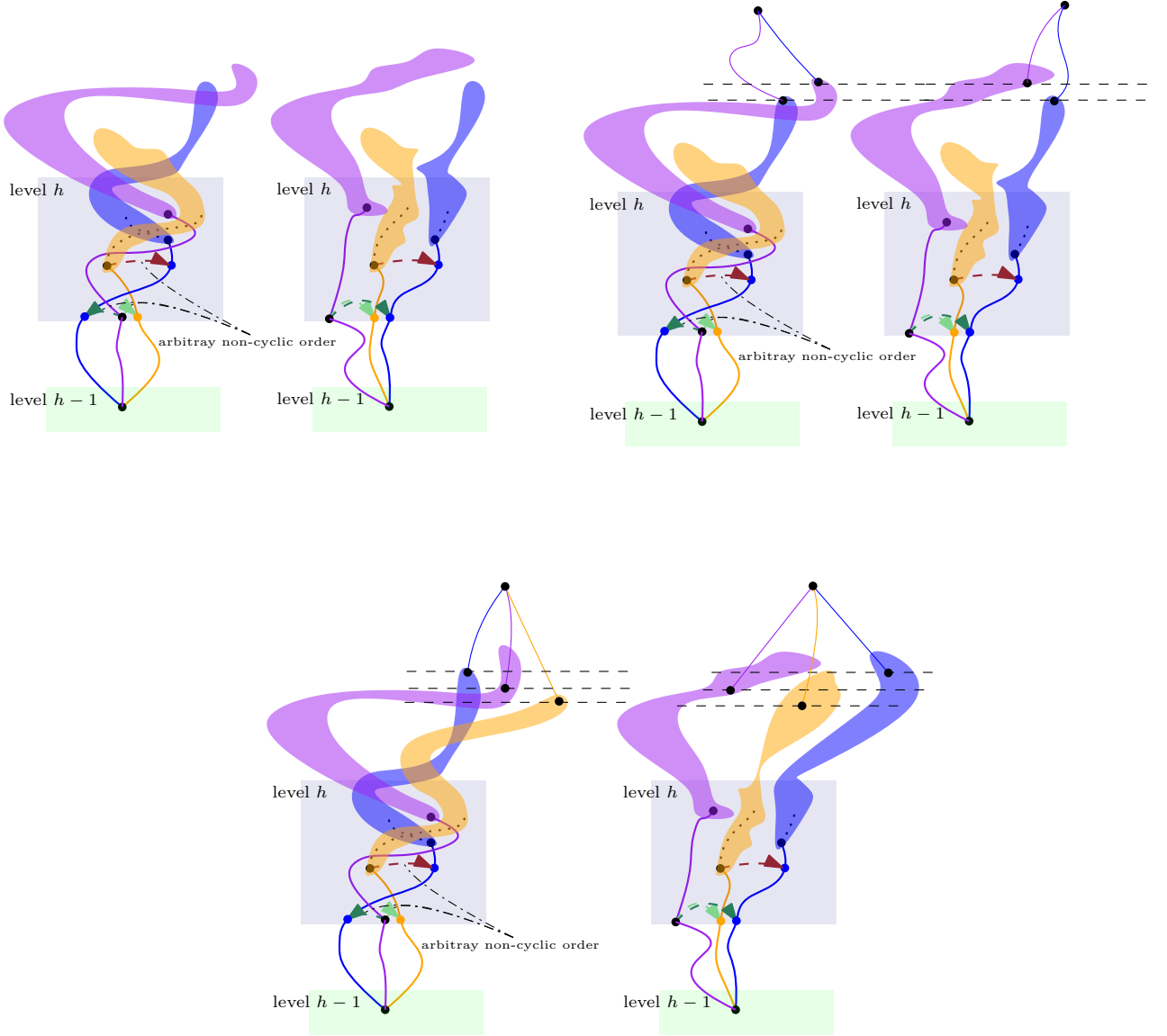


Figure 3.2.: in between even crossing edge

4. Conclusion

Summary and outlook.

Bibliography

Appendix

A. Appendix Section 1

ein Bild

Figure A.1.: A figure